

LAPORAN PRAKTIKUM PEKAN 4
STRUKTUR DATA

Queue

Disusun oleh:
Thaariq Salam
2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T
Asisten Praktikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas rahmat-Nya sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data mengenai materi "Queue". Laporan ini disusun untuk mendokumentasikan hasil pengerjaan serta pemahaman penulis terhadap mekanisme penyimpanan data FIFO (*First In, First Out*) dalam pemrograman Java.

Melalui praktikum ini, penulis telah mempelajari berbagai cara mengimplementasikan *Queue*, mulai dari penggunaan array statis, pemanfaatan kelas `ArrayList`, hingga penggunaan kelas bawaan `java.util.Queue`.

Saya menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan dan dukungan berbagai pihak. Oleh karena itu, saya menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan teknis selama pelaksanaan praktikum.

Saya menyadari sepenuhnya bahwa laporan praktikum ini masih jauh dari kata sempurna. Terdapat keterbatasan dalam analisis maupun teknis penulisan yang mungkin ditemukan. Oleh karena itu, saya sangat mengharapkan kritik yang membangun serta saran dari pembaca demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi saya pribadi maupun bagi pembaca yang ingin mendalami konsep pemrograman Java.

Padang, 29 April 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II.....	3
PEMBAHASAN.....	3
2.1 Class QueueArray_2511532022.java.....	3
2.1.1 Kode Program.....	3
2.1.2 Langkah Kerja dan Penjelasan Kode.....	5
2.1.3 Analisis.....	5
2.2 Class QueueArrayDriver_2511532022.java.....	6
2.2.1 Kode Program.....	6
2.2.2 Langkah Kerja dan Penjelasan Kode.....	6
2.2.3 Analisis.....	7
2.3 Class ReverseData_2511532022.java.....	8
2.3.1 Kode program.....	8
2.3.2 Langkah kerja dan Penjelasan Kode.....	9
2.3.3 Analisis.....	9
2.4 Class QueueLinkedList_2511532022.java.....	10
2.4.1 Kode program.....	10
2.4.2 Langkah kerja dan Penjelasan Kode.....	10
2.4.3 Analisis.....	11
2.5 Class IterasiQueue_251153022.java.....	11
2.5.1 Kode Program.....	11
2.5.2 Langkah kerja dan Penjelasan Kode.....	12
2.5.3 Analisis.....	12
BAB III.....	13
PENUTUP.....	13
3.1 Kesimpulan.....	13
TUGAS PEKAN 4.....	14
1. Class AntrianLoket_2511532022.java.....	14
1.1 Kode Program.....	14
1.2 Langkah Kerja dan Penjelasan Kode.....	16
1.3 Analisis.....	17

2. Class AntrianLoketDriver_2511532022.java.....	17
2.1 Kode Program.....	17
2.2 Langkah Kerja dan Penjelasan Kode.....	19
2.3 Analisis.....	19
3. Hasil Pengujian.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia informatika, pengelolaan data yang efisien merupakan kunci utama dalam pembangunan perangkat lunak yang optimal. Salah satu konsep dasar yang wajib dipahami adalah struktur data linier, di mana elemen-elemen data disusun secara berurutan. Di antara berbagai jenis struktur data, queue (antrian) menjadi sangat penting karena karakteristik uniknya yang menggunakan prinsip FIFO (*First In First Out*), yakni data yang pertama masuk akan menjadi yang pertama kali diproses atau dikeluarkan.

Penerapan *Queue* sangat luas dalam komputasi modern, mulai dari manajemen antrian proses pada sistem operasi, pengelolaan *buffer* pada transmisi data, hingga sistem layanan antrian digital. Dalam lingkungan Java, struktur ini dapat diimplementasikan secara manual menggunakan array statis untuk efisiensi memori, maupun menggunakan struktur yang lebih dinamis seperti *ArrayList* yang fleksibel dalam menangani jumlah data yang berubah-ubah.

Laporan ini disusun sebagai bentuk dokumentasi atas hasil pemahaman teknis terhadap konsep *Queue* yang telah dipraktikkan. Fokus utama dari laporan ini adalah penerapan prinsip FIFO dalam mengelola data objek terstruktur, guna memahami bagaimana alur data yang masuk dan keluar dapat dikelola secara sistematis sesuai dengan urutan kedatangannya.

1.2 Tujuan

1. Memahami prinsip kerja struktur data *Queue* dengan mekanisme FIFO (*First In First Out*) melalui simulasi antrian data.
2. Mampu mengimplementasikan *Queue* secara manual menggunakan Array dengan teknik *circular buffer* untuk mengoptimalkan penggunaan memori.

3. Mampu mengimplementasikan Queue secara dinamis dengan memanfaatkan *library java.util.Queue* dan *LinkedList*.
4. Mampu melakukan manipulasi data pada Queue, seperti membalikkan urutan data (*reverse*) dengan bantuan struktur data *Stack*.
5. Memahami teknik iterasi pada antrean menggunakan *interface* *Iterator* untuk mengakses elemen secara berurutan.

1.3 Manfaat

1. Meningkatkan kemampuan analisis dalam memilih struktur data yang tepat untuk kasus yang membutuhkan urutan pemrosesan yang adil (seperti sistem antrean).
2. Memahami logika pemrosesan data linier melalui operasi-operasi utama *Queue* seperti *enqueue (add)*, *dequeue (remove)*, dan *peek* (melihat data terdepan).
3. Mengasah kreativitas pemrograman dalam mengombinasikan dua struktur data yang berbeda (*Stack* dan *Queue*) untuk menyelesaikan persoalan manipulasi data.
4. Mampu mempraktikkan standarisasi penamaan dan struktur kode yang baik dalam pemrograman berorientasi objek menggunakan bahasa Java.

BAB II

PEMBAHASAN

2.1 Class QueueArray_2511532022.java

2.1.1 Kode Program

```
package pekan4_2511532022;

public class QueueArray_2511532022 {
    int front_2022, rear_2022, size_2022;
    int capacity_2022;
    int array_2022[];

    public QueueArray_2511532022(int capacity_2022) {
        this.capacity_2022 = capacity_2022;
        front_2022 = this.size_2022 = 0;
        rear_2022 = capacity_2022 - 1;
        array_2022 = new int[this.capacity_2022];
    }

    boolean isFull(QueueArray_2511532022 queue) {
        return (queue.size_2022 == queue.capacity_2022);
    }

    boolean isEmpty(QueueArray_2511532022 queue) {
        return (queue.size_2022 == 0);
    }

    void enqueue_2022(int item) {
        if (isFull(this))
            return;
        this.rear_2022 = (this.rear_2022 + 1) %
this.capacity_2022;
        this.array_2022 [this.rear_2022] = item;
        this.size_2022 = this.size_2022 + 1;
        System.out.println(item + " enqueue to queue ");
    }
}
```

```

int dequeue_2022() {
    if (isEmpty(this))
        return Integer.MIN_VALUE;
    int item = this.array_2022[this.front_2022];
    this.front_2022 = (this.front_2022 + 1 ) %
this.capacity_2022;
    this.size_2022 = this.size_2022 - 1;
    return item;
}

int front_2022() {
    if (isEmpty (this))
        return Integer.MIN_VALUE;

    return this.array_2022[this.front_2022];
}

int rear_2022() {
    if (isEmpty (this))
        return Integer.MIN_VALUE;

    return this.array_2022[this.rear_2022];
}
//mencetak elemen antrian
void display_2022() {
    int i;
    if (front_2022 == rear_2022) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    //kunjungi dari belakang dan cetak
    for (i = front_2022; i < rear_2022; i++) {
        System.out.printf(" %d <--", array_2022[i]);
    }
    System.out.printf(" %d", array_2022[rear_2022]);
    return;
}
}

```


2.1.2 Langkah Kerja dan Penjelasan Kode

1) Deklarasi Struktur:

Langkah awal adalah mendefinisikan variabel kontrol *front_2022* untuk menunjuk indeks elemen terdepan, *rear_2022* untuk elemen terakhir, dan *size_2022* untuk melacak jumlah data aktif agar pengecekan kondisi lebih cepat.

2) Inisialisasi Memori:

Melalui *constructor*: *public QueueArray_2511532022(int capacity_2022)*, array *array_2022* dipesan di memori sesuai kapasitas yang diinginkan. Di sini, *rear* diatur ke *capacity - 1* agar saat penambahan data pertama, indeks akan bergeser tepat ke angka 0.

3) Metode Enqueue (Tambah Data):

Pada baris *this.rear_2022 = (this.rear_2022 + 1) % this.capacity_2022;*, diterapkan logika *Circular Queue*. Operator modulo % memastikan bahwa jika *rear* sudah mencapai ujung array, ia akan berputar kembali ke indeks 0 selama masih ada ruang kosong (hasil pengecekan *isFull*).

4) Metode Dequeue (Hapus Data):

Penghapusan dilakukan dengan mengambil elemen pada *this.array_2022[this.front_2022]*, kemudian memperbarui posisi *front* menggunakan logika modulo yang sama agar posisi depan juga bergerak secara sirkular.

2.1.3 Analisis

Kelebihan utama dari penggunaan modulo ini adalah efisiensi. Tanpa sirkularitas, antrean akan cepat "penuh" secara semu karena indeks terus bergerak ke kanan, sementara ruang kosong di sebelah kiri (bekas data yang dihapus) tidak bisa digunakan kembali.

2.2 Class QueueArrayDriver_2511532022.java

2.2.1 Kode Program

```
package pekan4_2511532022;

public class QueueArrayDriver_2511532022 {
    public static void main(String[]args) {
        QueueArray_2511532022 queue = new
QueueArray_2511532022(1000);
        queue.enqueue_2022(10);
        queue.enqueue_2022(20);
        queue.enqueue_2022(30);
        queue.enqueue_2022(40);
        System.out.println("Item di depan " + queue.front_2022());
        System.out.println("item paling belakang " +
queue.rear_2022());
        System.out.println("tampilkan queue");
        queue.display_2022();
        System.out.println();
        System.out.println(queue.dequeue_2022 () + " dihapus dari
queue");
        System.out.println("item di depan: " + queue.front_2022());
        System.out.println("item di belakang: " +
queue.rear_2022());
        System.out.println("tampilkan queue setelah satu kata di
hapus");
        queue.display_2022();
    }
}
```

2.2.2 Langkah Kerja dan Penjelasan Kode

1) Instansiasi Objek:

Membuat objek *queue* dengan kapasitas 1000. Kapasitas besar ini digunakan untuk memastikan tidak terjadi *overflow* selama pengujian dasar.

2) Operasi Pengisian:

Memasukkan empat data integer (10, 20, 30, 40). Di sini, metode `queue.enqueue_2022(item)` dipanggil berulang kali, yang secara internal menggerakkan penanda *rear*.

3) Observasi Status:

Menggunakan `queue.front_2022()` untuk memverifikasi data di kepala antrean dan `queue.rear_2022()` untuk data di ekor. Pada tahap ini, output menunjukkan 10 di depan dan 40 di belakang.

4) Uji Prinsip FIFO:

Memanggil `queue.dequeue_2022()` dan mencetak hasilnya. Kode ini membuktikan bahwa angka 10 (yang pertama masuk) adalah yang pertama dihapus, sesuai aturan *First In First Out*.

2.2.3 Analisis

```
<terminated> QueueArrayDriver_2511532022 [Java Application]
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
item paling belakang 40
tampilkan queue
10 <-- 20 <-- 30 <-- 40
10 dihapus dari queue
item di depan: 20
item di belakang: 40
tampilkan queue setelah satu kata di hapus
20 <-- 30 <-- 40
```

Melalui metode `display_2022()`, terlihat jelas urutan data yang tersisa. Driver ini membuktikan bahwa logika pointer front dan rear pada kelas sebelumnya bekerja sinkron dalam menjaga urutan data.

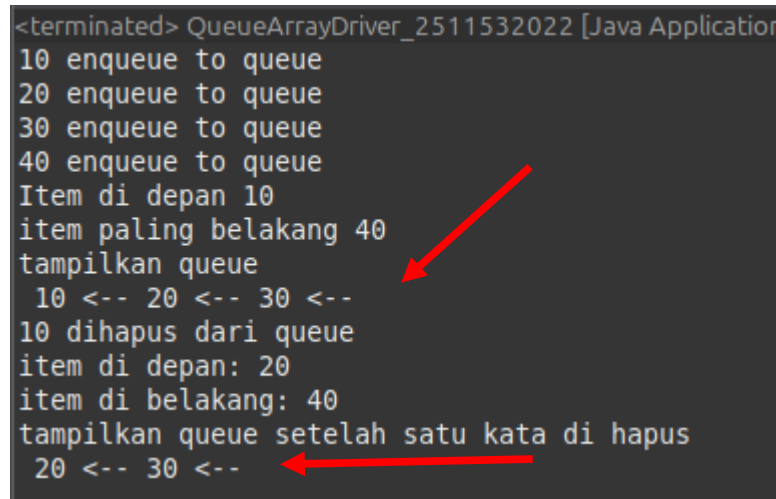
erdasarkan hasil pengujian pada `QueueArrayDriver_2511532022`, elemen terakhir (angka 40) tidak muncul pada *output console*. Hal ini dikarenakan batasan perulangan pada metode `display_2022()` yang hanya menggunakan kondisi $i <$

`rear_2022`, sehingga elemen pada indeks paling belakang tidak ikut tereksitasi ke layar.

Saya menambahkan baris baru pada class `QueueArray_251153202` agar dapat mencetak elemen yang berada pada `rear_2022` dapat tercetak.

```
System.out.printf(" %d", array_2022[rear_2022]);
```

Apabila tidak ditambahkan maka akan menghasilkan output sebagai berikut:



```
<terminated> QueueArrayDriver_2511532022 [Java Application]
10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
item paling belakang 40
tampilkan queue
 10 <-- 20 <-- 30 <--
10 dihapus dari queue
item di depan: 20
item di belakang: 40
tampilkan queue setelah satu kata di hapus
20 <-- 30 <--
```

Dapat dilihat *output* yang dihasilkan kode asli tidak mencetak angka-40 yang merupakan elemen `rear_2022` saat diperintahkan mencetak daftar *queue*.

2.3 Class ReverseData_2511532022.java

2.3.1 Kode program

```
package pekan4_2511532022;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
public class ReverseData_2511532022 {
    public static void main (String[]args) {
        Queue<Integer> q_2022 = new LinkedList<Integer>();
        q_2022.add(1);
        q_2022.add(2);
        q_2022.add(3);    // (1, 2, 3)
        System.out.println("Sebelum everse" + q_2022 );
        Stack<Integer> s_2022 = new Stack<Integer>();
        while (!q_2022 .isEmpty()) {
            s_2022 .push(q_2022.remove());
```

```

    }
    while (!s_2022 .isEmpty()) {
        q_2022.add(s_2022 .pop());
    }
    System.out.println("sesudah reverse = " +
q_2022 ); //[3, 2, 1]
    }
}

```

2.3.2 Langkah kerja dan Penjelasan Kode

1) Persiapan Dua Struktur:

Menyiapkan Queue (FIFO) dan Stack (LIFO) sebagai media penampung sementara.

2) Proses Migrasi Pertama:

Menggunakan loop *while (!q_2022.isEmpty())* untuk memindahkan data dari *Queue* ke *Stack* melalui perintah *s_2022.push(q_2022.remove())*. Pada tahap ini, urutan data di dalam *Stack* menjadi bertumpuk (1 di bawah, 3 di atas).

3) Proses Migrasi Kedua:

Mengeluarkan data dari *Stack* dengan *s_2022.pop()* dan memasukkannya kembali ke *Queue* dengan *.add()*.

2.3.3 Analisis

```

<terminated> ReverseData_2511532022 [Java Application]
Sebelum reverse = [1, 2, 3]
Sesudah reverse = [3, 2, 1]

```

Karena *Stack* mengeluarkan data terakhir terlebih dahulu (angka 3), maka saat data tersebut masuk kembali ke *Queue*, ia menjadi elemen pertama. Hasil akhirnya urutan [1, 2, 3] berubah menjadi [3, 2, 1]. Ini membuktikan pemahaman mendalam tentang perbedaan karakteristik operasional antara *Stack* dan *Queue*.

2.4 Class QueueLinkedList_2511532022.java

2.4.1 Kode program

```
package pekan4_2511532022;
import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532022 {
    public static void main (String[]args) {
        Queue<Integer> q_2022 = new LinkedList<>();
        // tambah elemen (0,1,2,3,4,5) ke antrian
        for (int i = 0; i < 6; i++)
            q_2022.add(i);
        //menampilkan isi antrian
        System.out.println(" Elemen antrian " + q_2022 );
        //menghapus kepala antrian
        int hapus = q_2022.remove();
        System.out.println(" Hapus elemen = " + hapus);
        System.out.println(q_2022);
        //untuk melihat antrian terdepan
        int depan = q_2022.peek();
        System.out.println("kepala antrian = " + depan);
        int banyak = q_2022.size();
        System.out.println("size antrian = " + banyak);
    }
}
```

2.4.2 Langkah kerja dan Penjelasan Kode

1) Automasi Input:

Menggunakan perulangan *for (int i = 0; i < 6; i++)* untuk memasukkan angka 0 sampai 5 ke dalam objek *q_2022* melalui metode *.add()*.

2) Observasi Kepala:

Menggunakan *q_2022.peek()* untuk melihat angka 0 di urutan terdepan tanpa menghapusnya dari koleksi.

3) Penghapusan Dinamis:

Menggunakan `q_2022.remove()` untuk membuang elemen kepala. Karena menggunakan `LinkedList`, memori yang digunakan oleh elemen yang dihapus akan langsung dikelola oleh *Garbage Collector* Java.

2.4.3 Analisis

```
<terminated> QueueLinkedList_2511532022 [Java Application]
Elemen antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala antrian = 1
Size antrian = 5
```

Perbedaan mencolok dengan versi Array adalah tidak adanya variabel `front` atau `rear` yang perlu dikelola secara manual. *Library* Java menyembunyikan kompleksitas tersebut, sehingga pengembang bisa fokus pada logika bisnis aplikasi.

2.5 Class IterasiQueue_251153022.java

2.5.1 Kode Program

```
package pekan4_2511532022;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
public class IterasiQueue_251153022 {
    public static void main (String[]args) {
        Queue<String> q_2022 = new LinkedList<>();

        q_2022.add("Pratikum");
        q_2022.add("Struktur");
        q_2022.add("Data");
        q_2022.add("Dan");
        q_2022.add("Algoritma");
        Iterator<String> iterator = q_2022 .iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next() + " ");
        }
    }
}
```

```
}  
}
```

2.5.2 Langkah kerja dan Penjelasan Kode

1) Pembuatan Koleksi:

Menggunakan `Queue<String> q_2022 = new LinkedList<>()`. Pemilihan `String` bertujuan untuk menguji kemampuan struktur data dalam menangani objek non-numerik.

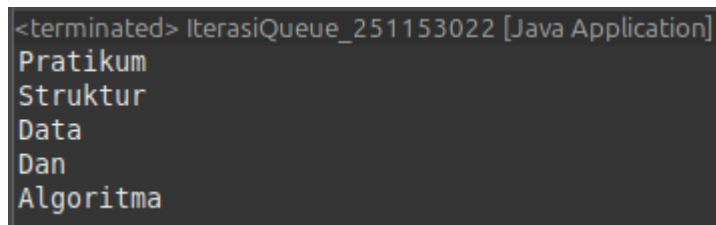
2) Penggunaan Iterator:

Kode `Iterator<String> iterator = q_2022.iterator();` memanggil objek pembantu yang bertugas menelusuri elemen.

3) Looping Terstruktur:

Menggunakan blok `while (iterator.hasNext())`. Fungsi `hasNext()` akan mengecek apakah masih ada elemen selanjutnya, sedangkan `iterator.next()` mengambil data tersebut dan menggeser kursor internal secara otomatis.

2.5.3 Analisis



```
<terminated> IterasiQueue_251153022 [Java Application]  
Pratikum  
Struktur  
Data  
Dan  
Algoritma
```

Teknik iterasi ini jauh lebih aman dan bersih dibandingkan menggunakan perulangan manual. Iterator memastikan bahwa urutan pencetakan tetap dari elemen terlama ke elemen terbaru tanpa risiko mengubah isi asli dari `Queue` tersebut.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum struktur data *Queue* yang telah dilaksanakan, dapat disimpulkan bahwa prinsip utama yang mendasari struktur ini adalah *First In First Out* (FIFO), di mana elemen yang pertama kali masuk akan menjadi yang pertama untuk diproses atau dihapus.

Melalui implementasi manual pada kelas *QueueArray_2511532022*, dipahami bahwa penggunaan logika *Circular Queue* dengan operator modulo sangat krusial untuk menjaga efisiensi memori dan mencegah terjadinya kesalahan status penuh pada *array* statis. Di sisi lain, penggunaan *library java.util.Queue* dan *LinkedList* memberikan fleksibilitas lebih tinggi karena sifatnya yang dinamis dalam menangani perubahan jumlah data tanpa perlu mendefinisikan kapasitas maksimal di awal.

Praktikum ini juga menunjukkan bahwa penelusuran elemen menggunakan *Iterator* dapat dilakukan tanpa merusak integritas data dalam antrean. Selain itu, manipulasi urutan data melalui kombinasi dengan struktur data *Stack* terbukti efektif untuk membalikkan urutan elemen berkat sifat LIFO yang dimilikinya.

Terakhir, hasil pengujian pada kelas *driver* menekankan pentingnya ketelitian dalam menyusun logika perulangan pada metode tampilan agar seluruh data, terutama elemen pada indeks *rear*, dapat terdeteksi dan ditampilkan secara akurat di layar.

TUGAS PEKAN 4

Topik : Implementasi Queue (FIFO) menggunakan java.util.Queue

Tema : Simulasi antrian pelanggan di loket pelayanan

1. Class AntrianLoket_2511532022.java

1.1 Kode Program

```
package pekan4_2511532022;
import java.util.Stack;

public class AntrianLoket_2511532022 {
    public int front_2022, rear_2022, size_2022;
    public int max_2022;
    public String[] queue_2022;

    public AntrianLoket_2511532022(int capacity_2022) {
        this.max_2022 = capacity_2022;
        this.front_2022 = this.size_2022 = 0;
        this.rear_2022 = capacity_2022 - 1;
        this.queue_2022 = new String[this.max_2022];
    }

    boolean isFull_2022(AntrianLoket_2511532022 q) {
        return (q.size_2022 == q.max_2022);
    }

    boolean isEmpty_2022(AntrianLoket_2511532022 q) {
        return (q.size_2022 == 0);
    }

    void enqueue_2022(String item_2022) {
        if (isFull_2022(this)) {
            System.out.println("Antrian penuh!");
            return;
        }
    }
}
```

```

    }
    this.rear_2022 = (this.rear_2022 + 1) % this.max_2022;
    this.queue_2022[this.rear_2022] = item_2022;
    this.size_2022 = this.size_2022 + 1;
    System.out.println(item_2022 + " berhasil ditambahkan ke
antrian");
}

String dequeue_2022() {
    if (isEmpty_2022(this)) {
        return "Antrian kosong!";
    }
    String item_2022 = this.queue_2022[this.front_2022];
    this.front_2022 = (this.front_2022 + 1) % this.max_2022;
    this.size_2022 = this.size_2022 - 1;
    return item_2022;
}

void display_2022() {
    if (isEmpty_2022(this)) {
        System.out.println("Antrian kosong.");
        return;
    }
    System.out.println("Isi antrian saat ini:");
    int count = 0;
    int i = this.front_2022;
    while (count < this.size_2022) {
        System.out.println((count + 1) + ". " +
queue_2022[i]);
        i = (i + 1) % this.max_2022;
        count++;
    }
}

void reverse_2022() {
    if (isEmpty_2022(this)) return;

    Stack<String> s_2022 = new Stack<>();
    int count = 0;

```

```

        int i = this.front_2022;

        // Pindahkan elemen ke stack untuk membalikkan urutan
        while (count < this.size_2022) {
            s_2022.push(queue_2022[i]);
            i = (i + 1) % this.max_2022;
            count++;
        }

        // Kembalikan elemen ke queue (urutan otomatis terbalik
        karena LIFO)
        int j = this.front_2022;
        while (!s_2022.isEmpty()) {
            queue_2022[j] = s_2022.pop();
            j = (j + 1) % this.max_2022;
        }
        System.out.println("Antrian berhasil dibalik (Reverse).");
    }
}

```

1.2 Langkah Kerja dan Penjelasan Kode

1. Inisialisasi Struktur:

Variabel *front_2022* dan *rear_2022* didefinisikan untuk mengelola posisi indeks, sementara variabel *size_2022* digunakan sebagai indikator utama jumlah pelanggan aktif guna menghindari ambiguitas antara kondisi antrean kosong dan penuh.

2. Implementasi Enqueue:

Melalui perintah *this.rear_2022 = (this.rear_2022 + 1) % this.max_2022*, indeks belakang bergerak secara sirkular. Hal ini memastikan bahwa ruang array yang telah ditinggalkan oleh pelanggan sebelumnya dapat digunakan kembali tanpa perlu menggeser seluruh elemen.

3. Implementasi Dequeue:

Metode ini mengambil data dari *front_2022* dan memperbarui indeks depan menggunakan logika modulo yang sama. Pengurangan *size_2022* dilakukan secara eksplisit untuk menjaga akurasi pemantauan kapasitas.

4. Logika Pembalikan (*Reverse*):

Teknik ini menggunakan bantuan objek *Stack<String> s_2022*. Data dipindahkan dari antrean ke tumpukan melalui loop *while (count < this.size_2022)* dan dikembalikan lagi ke antrean menggunakan loop *while (!s_2022.isEmpty())*. Sifat LIFO pada *stack* secara otomatis membalikkan urutan nama pelanggan.

1.3 Analisis

Penggunaan metode manual dengan pengecekan *isFull_2022(this)* dan *isEmpty_2022(this)* memberikan kontrol penuh atas alokasi memori array statis. Implementasi *reverse_2022* terbukti sangat efektif karena proses pengisian kembali data ke array hanya bergantung pada ketersediaan data di dalam *Stack* melalui pengecekan kondisi kosong secara langsung.

2. Class *AntrianLoketDriver_2511532022.java*

2.1 Kode Program

```
package pekan4_2511532022;
import java.util.Scanner;

public class AntrianLoketDriver_2511532022 {
    public static void main(String[] args) {
        AntrianLoket_2511532022 loket_2022 = new
AntrianLoket_2511532022(10);
        Scanner sc_2022 = new Scanner(System.in);
        int pilih_2022;
        System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
        System.out.println("1. Tambah Antrian");
        System.out.println("2. Hapus Antrian");
```

```

System.out.println("3. Tampilkan Antrian");
System.out.println("4. Reverse");
System.out.println("5. Keluar");

do {

    System.out.print("\nPilih menu: ");
    pilih_2022 = sc_2022.nextInt();
    sc_2022.nextLine();

    switch (pilih_2022) {
        case 1:
            System.out.print("Masukkan nama pelanggan: ");
            String nama_2022 = sc_2022.nextLine();
            loket_2022.enqueue_2022(nama_2022);
            break;
        case 2:
            String hasil_2022 = loket_2022.dequeue_2022();
            if (!hasil_2022.equals("Antrian kosong!")) {
                System.out.println(hasil_2022 + " telah
dilayani.");
            } else {
                System.out.println(hasil_2022);
            }
            break;
        case 4:
            loket_2022.reverse_2022();
            loket_2022.display_2022();
            break;
        case 3:
            loket_2022.display_2022();
            break;
        case 5:
            System.out.println("Program selesai.");
            break;
        default:
            System.out.println("Menu tidak tersedia.");
    }
} while (pilih_2022 != 5);

```

```

        sc_2022.close();
    }
}

```

2.2 Langkah Kerja dan Penjelasan Kode

1. Manajemen Input:

Menggunakan *Scanner sc_2022* untuk menerima instruksi angka menu dan input teks berupa nama pelanggan. Penggunaan *sc_2022.nextLine()* setelah *nextInt()* sangat krusial untuk membersihkan karakter *newline* agar input nama tidak terlewat.

2. Sistem Menu *Switch-Case*:

Program menyediakan lima opsi utama, termasuk penambahan, penghapusan, dan pembalikan antrian. Opsi ke-4 secara otomatis menjalankan fungsi *reverse_2022()* diikuti dengan *display_2022()* untuk menunjukkan hasil perubahan urutan secara langsung.

3. Output Informasi:

Menggunakan logika percabangan *if-else* saat melakukan *dequeue* untuk membedakan antara pesan sukses (pelanggan dilayani) dan pesan peringatan jika antrian sedang dalam keadaan kosong.

2.3 Analisis

Pemisahan kode menjadi dua file (Logika dan *Driver*) memudahkan proses pelacakan kesalahan (*debugging*). *Driver* hanya fokus pada interaksi dan validasi menu, sementara integritas data antrian tetap dikelola secara terpusat di dalam kelas *AntrianLoket*. Penggunaan akhiran *_2022* pada seluruh variabel dan metode memastikan konsistensi gaya pemrograman di sepanjang laporan praktikum ini.

3. Hasil Pengujian

```
<terminated> AntrianLoketDriver_2511532022 [Java Application]

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1
Masukkan nama pelanggan: Thaariq
Thaariq berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan nama pelanggan: Joko
Joko berhasil ditambahkan ke antrian

Pilih menu: 3
Isi antrian saat ini:
1. Thaariq
2. Joko

Pilih menu: 4
Antrian berhasil dibalik (Reverse).
Isi antrian saat ini:
1. Joko
2. Thaariq

Pilih menu: 2
Joko telah dilayani.

Pilih menu: 3
Isi antrian saat ini:
1. Thaariq

Pilih menu: 5
Program selesai.
```